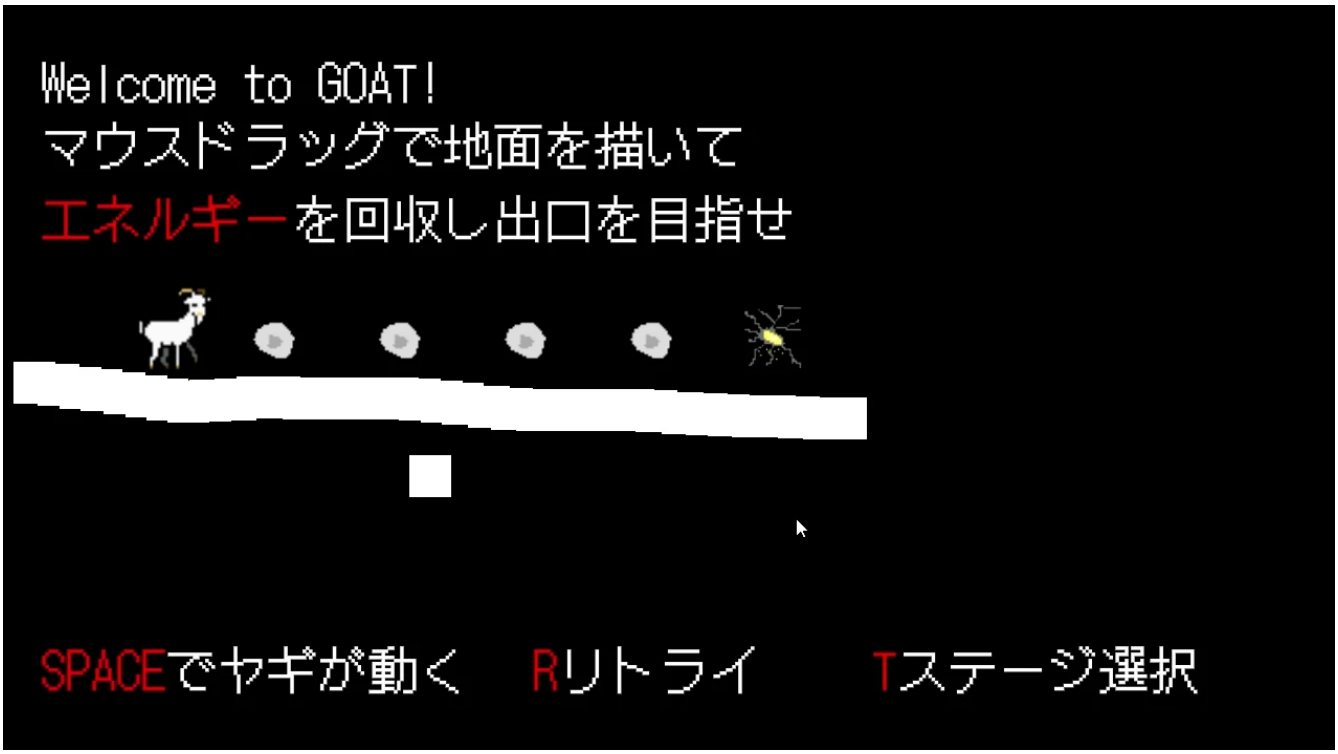
GOAT

描いた道で、自動で歩くヤギをゴールへ導く2Dパズルゲーム



ゲーム画面：ドラッグで足場を描き、エネルギーを回収して出口へ進む。

バンダイナムコスタジオ杯 Siv3Dゲームジャム2025　最優秀賞

制作期間：2025/10/18 - 11/2（約2週間）　/　使用技術：C++・Siv3D

01　担当範囲

アニメーション以外のメインシステム全般を担当。リーダー・リードプログラマー・プランナーを兼任しました。

入力・地形	マウス入力による足場生成、編集／再生モードの切り替え
ヤギの制御	自動移動、重力、着地・天井判定、段差越え、ジャンプ制限
進行・ギミック	収集物、ゴール開放・到達判定、ワープ、リトライ
担当外	アニメーション（本資料の技術解説には含めません）

作品紹介・ダウンロード案内（GOAT）

公式受賞結果

描いた道を、歩ける地形にする

担当：足場生成 / 編集・再生の切り替え / 足場データと移動処理の接続

01 入力を当たり判定のある足場へ変換

ドラッグ中のカーソル位置から40×40の矩形を生成し、足場の配列に追加します。この配列を描画とヤギの衝突判定に利用し、プレイヤーがつくった道をゲーム内で機能する地形にしています。

ドラッグ入力 > **足場の配列** > **描画・衝突判定**

02 生成位置の間隔を管理

直前に生成した位置を保持し、そこから20pxを超えてカーソルが移動したときに足場を追加します。同じ位置付近で毎フレーム追加し続けることを抑える条件です。

```
if (!lastDrawPos ||
    lastDrawPos->distanceFrom(current) > 20)
{
    stage.platforms << RectF{ Arg::center(current), 40, 40 };
    lastDrawPos = current;
}
```

03 つくる・動かす・やり直すを分ける

編集モードでは道を作成し、SPACEでヤギの移動を開始。Rで足場や取得状況を初期化して描き直せます。道を考え、挙動を確かめ、再挑戦する流れを支える仕組みです。

改善余地：現在は入力点の間を補間していないため、高速なドラッグで生じる足場の隙間への対応が課題です。

実装参照：GameScene.hpp / update()・resetPlayer()（入力処理のコード抜粋）

地形に応じた、自動移動の制御

担当：重力 / 接地・天井判定 / 段差越え / 条件付き自動ジャンプ

01 小さな段差とジャンプを分けて処理

次の横移動位置で足場に重なるかを調べ、最大8pxまで1px刻みで上方向へ位置をずらして確認します。小さな段差を越えられない場合は、接地状態とジャンプ回数などの条件から行動を決めます。

1 **移動先の足場との重なりを確認**

重なりがなければ、そのまま横移動

2 **0〜8pxの上方向補正を試す**

対象の足場と重ならない位置があれば、段差を越える

3 **越えられなければジャンプ条件を確認**

接地中・ジャンプ中でない・回数上限未満なら跳ぶ

処理の流れを説明する模式図（実際の画面ではありません）

02 衝突方向とジャンプ中の状態を管理

落下時は着地、上昇時は天井への衝突を判定し、位置と縦方向の速度を補正します。ジャンプ上昇中は横移動を止め、下降に入ると再開する制御を組み合わせています。

技術的な担当：Siv3Dの矩形・交差判定を利用し、位置補正と移動状態の制御を実装。図形判定の結果を位置・速度へ反映し、遊びに合わせた移動・衝突応答を組み立てています。

実装参照：Player.hpp / update()（段差補正・着地判定・ジャンプ制御）

共通の仕組みから、遊びの違いをつくる

担当：ステージ配置の仕組み / ステージごとの制約 / ギミック連携

01 数字の配置でステージを組み立てる

Grid<int32>の値で収集物・障害物・ゴール・ワープを指定し、共通の処理で読み取ります。各ステージは配置データを差し替える構成とし、遊びの条件を調整できる土台を用意しました。

配置データ → 共通の読み取り処理 → ステージ

1：収集物 2：障害物 4：閉じたゴール 5／6：ワープ入口／出口

02 ジャンプ制限と配置で、攻略の考え方を変える

ステージ1	**上限100回**	基本操作と収集・ゴールのルールを提示
ステージ2	**上限2回**	限られたジャンプを意識する構成
ステージ3	**上限0回**	ワープ・画面端での反転を使う構成

前のステージで学んだ動作を踏まえ、次に何を理解してほしいかを考えてギミックを実装する方針です。テンプレートと設計意図をチームに共有し、試作した内容を精査することを大切にしています。

03 目指した体験と、今後の改善

道筋を一つに限定せず、プレイヤーの試行錯誤が攻略につながる遊びを目指しました。今後の改善候補は、入力軌跡の補間とフレーム時間に依存しない移動計算です。操作感の安定性を高める余地があります。

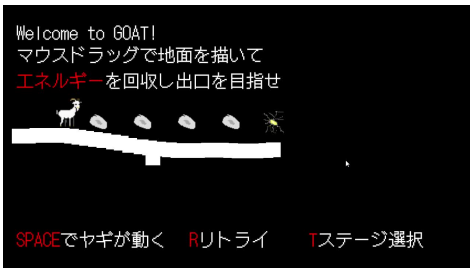
ポートフォリオ：GOATの作品紹介

実装参照：Stage.hpp / CreateMapStage0〜5()、GameScene.hpp / ステージ別設定

実際の動作で、担当箇所を確認

同封動画：GOAT_技術解説付きプレイ動画.mp4（約79秒・音声付き）

下の画像、またはページ下部のリンクをクリックすると、ブラウザーで動画が開きます。動画の字幕で担当システムを説明しています。アニメーションは担当外です。



00:04 - 00:16

道の描画からゴールまで

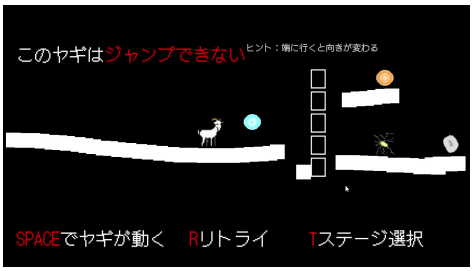
自由に描いた足場を、ヤギの移動・衝突判定に接続。収集物の回収に応じてゴールを開放します。



00:18 - 00:35

段差・ジャンプと移動状態

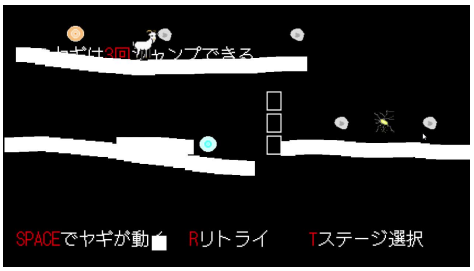
地形との衝突に応じて移動を制御。ジャンプ回数の制約がある中で、描いた道を攻略につなげます。



00:36 - 00:53

ワープを使った攻略

ジャンプできない条件でワープを利用。収集・ワープ・ゴール判定を連携させています。



00:54 - 01:15

異なる配置でも共通処理で動作

足場生成・移動処理を共通化し、配置や制約の違いによって別の攻略を考えるステージを構成します。

▶ プレイ動画を見る（約79秒・ブラウザーで再生）

okayu2345.github.io/videos/goat-gameplay.mp4